Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative

C Programming Lab - Assignment 1 - Question 1 (50 Marks)

FILE 1 of 2 : ASSIGNMENT REPORT (Tasks 1 - 12)

Student	R. Surya
Register No.	1924260114C
Institution	SIMATS Engineering
Compiler / platform	gcc 13.3.0, Ubuntu 24.04 (x86-64, LP64: int = 4 B, long = 8 B)
Companion file	FILE 2 : Source code, input files, execution output, README.md (everything referred to below as 'File 2 - Section X')

Contents of this report

Assumptions - Fig.1 UML - Fig.2 Flowchart - Task 1 Problem and data - Task 2 Data types - Task 3 Constants - Task 4 Macros - Task 5 Header files and conditional compilation - Task 6 Input/output - Task 7 Type conversion - Task 8 Compilation process - Task 9 Implementation and testing (with pseudocode, test tables) - Task 10 Expectation vs execution - Task 11 Final design - Task 12 Smart-meter redesign (A to D) - Marks map - Viva preparation.

Assumptions (the question does not fix the tariff, so these are stated and used consistently)

- Billing is per meter reading cycle. Units consumed = current reading - previous reading (kWh); meter register is a **16-bit unsigned** counter (0..65535) that wraps to 0 after 65535.
- **Telescopic slab tariff** (each slab's rate applies only to units inside it): units 1-100 at RATE1 = 150 paise (Rs 1.50); 101-200 at RATE2 = 200 paise (Rs 2.00); above 200 at RATE3 = 300 paise (Rs 3.00). Task 3 changes RATE2 to 230 paise, as the question demands.
- **Fixed charge by category** (stored in paise): Domestic = 5000 (Rs 50), Commercial = 15000 (Rs 150), Agriculture = 2500 (Rs 25). Total bill = energy charge + fixed charge. A zero-unit bill still pays the fixed charge.
- All money is held as an **integer number of paise** (100 paise = Rs 1). Household numbers are 1..999999. Category codes: 0 Domestic, 1 Commercial, 2 Agriculture.
- Rollover is declared by the meter reader with a flag (y/n). A decrease without the flag is invalid input; the flag with current >= previous is also invalid.
- District scale = 800,000 households (the largest size named in Task 10). Smart-meter assumed maximum connected load = 10 kW (Task 12).

Fig. 1 and Fig. 2 - UML and algorithm

Fig. 1 UML class / module diagram of the billing system

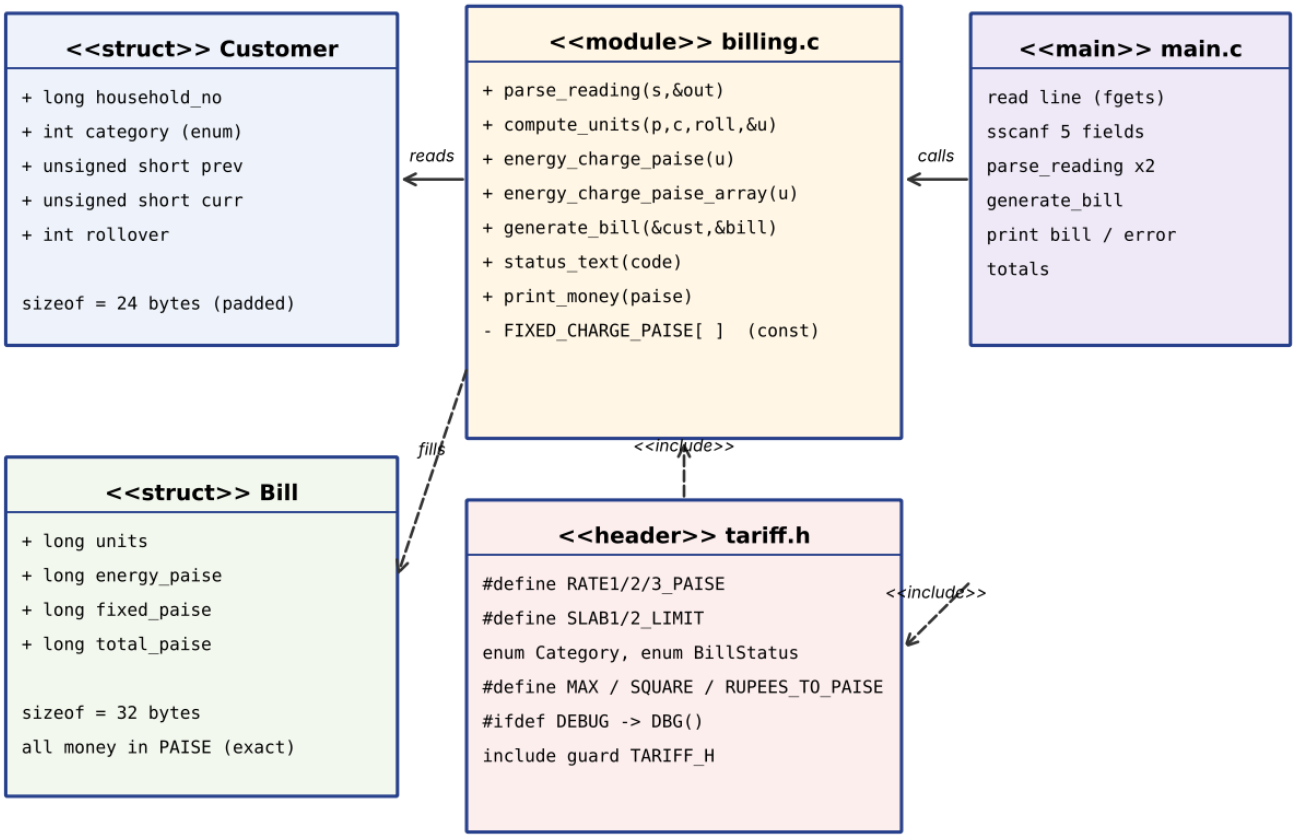
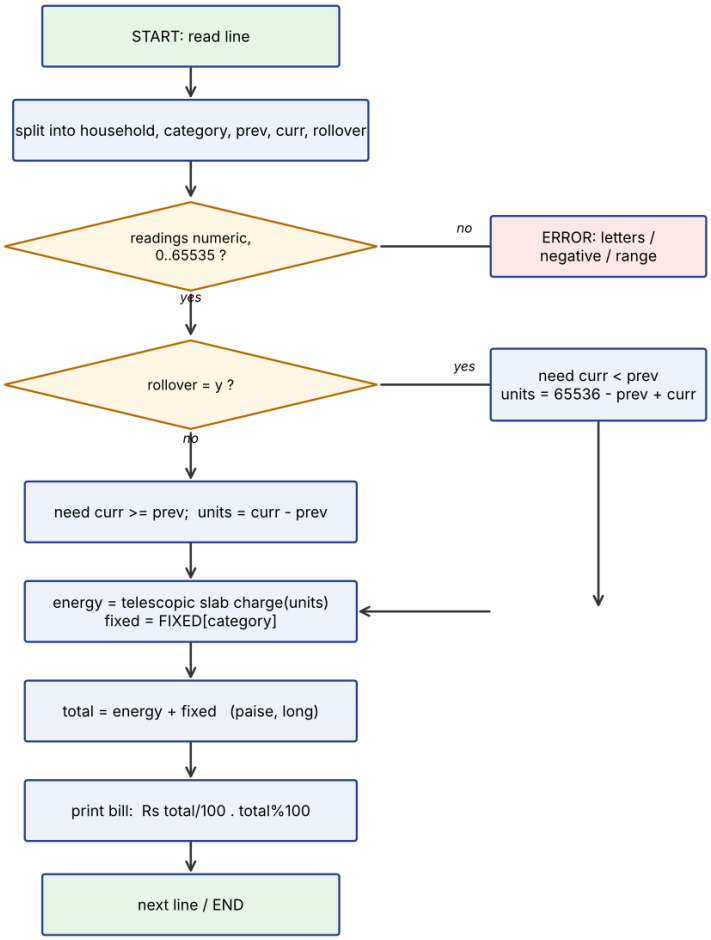


Fig. 2 Flowchart of the billing algorithm



TASK 1 - Understand the Problem and Identify the Data (4 marks)

The co-operative must turn two meter readings into a correct, auditable bill. The program must **store** the identity and readings, and **compute** units, energy charge, fixed charge and total.

1.1 Inputs, outputs, constants

Item	Kind	Range / rule	Justification
Household number	Input	1..999999 (6 digits)	Identifier only - never used in arithmetic; needs more than 16 bits (65535 max), so 32+ bit integer.
Category	Input	0,1,2 (Domestic, Commercial, Agriculture)	Small fixed set -> enum (readable names, compiler-visible values); selects the fixed charge.
Previous reading	Input	0..65535	Value shown by a 16-bit register; non-negative integer, so unsigned short matches the hardware exactly.
Current reading	Input	0..65535	Same register; may be smaller than previous only if it wrapped (rollover).
Rollover flag	Input	y / n	Needed to tell a wrap (65535->0) apart from a faulty/decreasing reading.
Units consumed	Output	0..65535 per cycle	curr - prev, or 65536 - prev + curr after a wrap. Held in a long so later multiplication by the rates cannot overflow 16 bits.
Energy charge	Output	up to ~19.6 million paise	units x slab rate; integer paise to avoid rounding error (long).
Fixed charge	Output	2500 / 5000 / 15000 paise	Constant per category; const array indexed by the enum.
Total bill	Output	energy + fixed	long paise; printed as Rs total/100 . total%100.
Constants	Constant	RATE1-3, SLAB1-2 limits, METER_MODULUS 65536, FIXED[]	Defined once in tariff.h so a tariff revision touches one place.

1.2 Possible negative / fractional / large values and rules

- **Negative:** a reading such as -5 is physically impossible -> rejected (status ERR_NEGATIVE). A negative *difference* (current < previous) is legal only with rollover.
- **Fractional:** the meter register is an integer counter, so '12.5' is rejected as non-numeric. Fractional *money* (e.g. 33 x Rs 1.505 = Rs 49.665) is avoided by working in paise (Task 7).
- **Large:** readings above 65535 are rejected (ERR_RANGE). One bill can reach about 19.6 lakh paise; a district total over 800,000 bills needs 64-bit (Task 10).
- **Rules:** (1) units = curr - prev; (2) if curr < prev and rollover flagged, units = 65536 - prev + curr; (3) slab charge is telescopic; (4) total = energy + fixed; (5) every input is validated before computing.
- **Assumptions:** listed on page 1; a 16-bit register wraps at most once per cycle.

1.3 Integer readings vs text-based readings (time and space complexity)

Aspect	Integer reading (unsigned short)	Text reading (char[6])
Space per reading	2 bytes, O(1)	up to 5 digits + '\0' = 6 bytes, O(d) where d = digits
Space for n households (2 readings)	4n bytes = O(n)	12n bytes = O(n d)
Subtract (units)	1 machine instruction, O(1)	must convert both strings (strtol) O(d) first, then subtract
Compare	numeric compare, correct	lexicographic compare is WRONG ('900' > '1000'); must convert or pad
Validation	validated once on input, afterwards always a legal number	garbage ('12x', '-5') can survive inside the data and fail later
Overall time for n bills	O(n)	O(n d) = O(n) since d <= 5, but with a larger constant
Verdict	**Chosen**: smaller, faster, matches the 16-bit hardware and its wrap-around	Only useful for display / raw input before validation

TASK 2 - Choose the Data Types (4 marks)

Evidence: File 2 - Section 'task2_types.c' and its execution output. Sizes below were obtained with `sizeof`, ranges from `limits.h`/`float.h`.

2.1 Selected types, sizes and ranges (this machine)

Data item	C type	sizeof	Range (from limits.h)	Reason
Household no.	long	8	-9.22e18 .. 9.22e18	6-digit id; long is guaranteed >= 32 bits (an int may be only 16).
Category	int / enum Category	4	0..2	named constants; int-sized.
Meter reading	unsigned short	2	0 .. 65535 (USHRT_MAX)	exactly the 16-bit register.
Units, charges (paise)	long	8	LONG_MAX 9223372036854775807	arithmetic in a wider type than the 16-bit reading, no overflow in one bill.
District totals	long long / int64_t	8	LLONG_MAX 9.22e18	guaranteed 64 bits on every platform.
(rejected) money as float	float	4	FLT_MAX 3.4e38, FLT_DIG 6	only ~6-7 significant digits: loses paise.
(rejected) money as double	double	8	DBL_MAX 1.8e308, DBL_DIG 15	binary fractions: 0.01 is not exact.

Other sizes measured: char 1, short 2, int 4, long 8, long long 8, float 4, double 8, int32_t 4, int64_t 8. **Portability note:** on Windows sizeof(long) is 4; the C standard only promises int >= 16 bits, long >= 32 bits, long long >= 64 bits (see Task 10).

2.2 Why store money as integer paise, not float rupees

- Decimal fractions such as 0.01, 0.1, 1234.57 have **no exact binary representation**; every float/double addition adds a tiny error that accumulates.
- Integer arithmetic is exact (until overflow), so every bill and every total is reproducible and auditable - essential for money.
- Comparisons are reliable (`0.1 + 0.2 == 0.3` is false in double; integer 30 == 30 is true).

2.3 Float precision demonstration - adding 4,000 bills of Rs 1234.57 (measured)

Method	Result	Error vs exact Rs 49,38,280.00
float rupees	Rs 4938160.0000	**-120.00 rupees** (12,000 paise lost)
double rupees	Rs 4938280.000000	-0.00000007 (invisible here, grows with n)
integer paise (long)	493828000 paise = Rs 4938280.00	0 (exact)

Further proof (task2_types.c): `16777216.0f + 1.0f == 16777216.0f` is true - a float cannot even count single paise above 2^{24}. Adding 0.01 twenty million times in double gives 200000.000034 instead of 200000; the integer sum is exactly Rs 200000.00.

2.4 16-bit unsigned meter register rollover after 65535

An `unsigned short` holds 0..65535. C defines unsigned arithmetic **modulo 2^{16} = 65536**, so 65535 + 1 becomes 0 (no error, no warning). The meter hardware behaves the same way, so after reading 65535 the next reading is 0. Billing must therefore detect the wrap:

```
unsigned short r = 65535;  r = r + 1;          /* r becomes 0                          */
unsigned short prev = 65530, curr = 4;         /* meter went 65530 -> 65535 -> 0 -> 4   */
long units = (curr >= prev) ? (long)curr - prev   /* normal case                       */
                            : 65536L - prev + curr; /* rollover case = 65536-65530+4 = 10  */
unsigned short u16 = (unsigned short)(curr - prev); /* modular arithmetic also gives 10   */
```

Output explained (File 2): `r = 0`; naive `curr - prev` (computed as int) = -65526 (wrong); `(unsigned short)(curr-prev)` = 10 and `65536 - prev + curr` = 10 (both correct). A 16-bit register can display only 65536 distinct readings, so a household using 264 kWh/month would wrap every ~250 months - but a smart meter reporting watt-hours would wrap four times per month (Task 12).

2.5 double rupees vs integer paise - time and space complexity

Criterion	double rupees	long paise
Accuracy	approximate; error grows with number of operations	exact
Time per operation	O(1); FP add (latency 3-5 cycles)	O(1); integer add (1 cycle)
Time for n bills	O(n)	O(n)
Measured: 20,000,000 additions	0.011 s	0.011 s

Criterion	double rupees	long paise
Space per amount	8 bytes	8 bytes (4 bytes int32 suffices for one bill but not totals); O(1) each, O(n) for n bills
Printing	%.2f rounds silently	Rs p/100 . p%100 exact

TASK 3 - Define the Constants (3 marks)

Tariff values and limits are defined in `tariff.h` (File 2) as macros: RATE1_PAISE 150L, RATE2_PAISE 200L, RATE3_PAISE 300L, SLAB1_LIMIT 100L, SLAB2_LIMIT 200L, METER_MODULUS 65536L; the fixed charges are a `const` array and the categories an `enum`.

3.1 #define, const and enum

- **#define** - a preprocessor text substitution done before compilation; no type, no storage, no address. Example: `#define RATE1_PAISE 150L`.
- **const** - a normal typed variable the compiler will not let you modify; has storage and an address. Example: `static const long K_RATE1 = 150L;`. In C a const int is *not* a constant expression (cannot be a case label or a static array size).
- **enum** - a set of named integer constants, compile-time values usable in case labels and arrays; ideal for related integer codes (CAT_DOMESTIC=0, CAT_COMMERCIAL=1, CAT_AGRICULTURE=2, CAT_COUNT).

3.2 Comparison

Feature	#define macro	const variable	enum
Type checking	none (raw text, suffix such as L must be written by hand)	full type checking	int-compatible; names grouped into a type
Scope	from #define to end of file (or #undef); ignores block scope	normal C scope (block/file/static)	normal scope (enumerators share the enclosing scope)
Debugging	name invisible to debugger (replaced by literal)	visible, can print/watch	visible: debugger shows CAT_DOMESTIC
Memory	none (immediate literal in code)	occupies memory (measured: sizeof = 8, has an address)	none for values (sizeof(enum) = 4 for a variable)
Usable in case / array size	yes	no in C	yes
Changing at build time	gcc -DRATE2_PAISE=230 (no edit)	must edit source (measured: K_RATE2 stayed 200)	edit source
Best for	tariff numbers, include guards, debug switches	fixed tables (FIXED_CHARGE_PAISE[])	category codes, status codes

3.3 Naming conventions followed

Macros and global constants in UPPER_SNAKE_CASE with unit suffix (`RATE1_PAISE`, `SLAB1_LIMIT`); enumerators prefixed by their type (`CAT_`, `ERR_`); file-local const with a k/K_ prefix; types in PascalCase (`Customer`, `Bill`); functions in snake_case; header guard `TARIFF_H`.

3.4 Billing function using the macros

```
long energy_charge_paise(long units)
{
    if (units <= SLAB1_LIMIT)
        return units * RATE1_PAISE;
    else if (units <= SLAB2_LIMIT)
        return SLAB1_LIMIT * RATE1_PAISE + (units - SLAB1_LIMIT) * RATE2_PAISE;
    else
        return SLAB1_LIMIT * RATE1_PAISE + (SLAB2_LIMIT - SLAB1_LIMIT) * RATE2_PAISE
             + (units - SLAB2_LIMIT) * RATE3_PAISE;
}
```

Full program: task3_const.c and billing.c (File 2). Output for tariff 150/200/300 and fixed Rs 50 (domestic):

3.5 RATE2_PAISE changed to 230 and bills verified

The change needs **no source edit**: tariff.h wraps each rate in `#ifndef`, so `gcc -DRATE2_PAISE=230` overrides it. Hand calculation (domestic, fixed 5000 paise): 150 units = 15000 + 50 x 230 + 5000 = **31500**; 200 units = 15000 + 100 x 230 + 5000 = **43000**; 250 units = 15000 + 23000 + 50 x 300 + 5000 = **58000**; 300 units = 15000 + 23000 + 30000 + 5000 = **73000**. The program printed exactly these values (File 2, Task 3 output). Bills with units <= 100 are unchanged (12500, 20000) as expected.

Units (domestic)	RATE2 = 200: paise	RATE2 = 230: paise	Hand check (230)	Match
50	12500	12500	7500 + 5000 = 12500	yes
100	20000	20000	15000 + 5000	yes
150	30000	31500	15000 + 11500 + 5000	yes
200	40000	43000	15000 + 23000 + 5000	yes
250	55000	58000	15000 + 23000 + 15000 + 5000	yes

Units (domestic)	RATE2 = 200: paise	RATE2 = 230: paise	Hand check (230)	Match
300	70000	73000	15000 + 23000 + 30000 + 5000	yes

3.6 Constants as macros vs const variables vs enums - conclusion

Tariff **rates and limits** -> macros (compile-time constants usable anywhere, overridable with -D). **Per-category table** -> const array (indexed at run time). **Category and error codes** -> enum (readable, debugger-visible, and CAT_COUNT sizes the table automatically). Task 12 shows that when tariffs must change without recompiling none of the three is enough - values move to a configuration file.

TASK 4 - Use Macros Safely (4 marks)

4.1 Macros for repeated calculations

```
#define RUPEES_TO_PAISE(r)  ((long)((r) * 100.0 + 0.5))
#define MAX(a, b)           (((a) > (b)) ? (a) : (b))
#define SQUARE(x)           ((x) * (x))
```

Output: RUPEES_TO_PAISE(12.34) = 1234 paise, RUPEES_TO_PAISE(99.99) = 9999 paise, MAX(150, 230) = 230. Every parameter and the whole body are parenthesised.

4.2 Macro expansion with gcc -E (File 2, Task 4 output)

```
$ gcc -E macro_expand.c | tail -n 2
long demo(long units) { return ((units + 1) * (units + 1)) + (((units) > (100)) ? (units) : (100)) + ((long)((2.5) * 100.0 + 0.5)); }
long rate(void) { return 200L; }
```

The preprocessor has replaced SQUARE, MAX, RUPEES_TO_PAISE and RATE2_PAISE by plain text; the compiler never sees the macro names.

4.3 Why SQUARE(x) without parentheses is wrong

With `#define BAD_SQUARE(x) x * x`, the call `BAD_SQUARE(2+3)` expands to `2+3 * 2+3`; since * binds tighter than +, the value is 2 + 6 + 3 = **11**, not 25 (measured: 11). The safe version `((x)*(x))` gives 25. Similarly `10 + BAD_MAX(1,2)` (a > b ? a : b) expands to `10 + 1 > 2 ? 1 : 2` = 1 instead of 12 (measured: 1).

4.4 MAX(a, b) may evaluate arguments more than once

`MAX(i++, j++)` expands to `((i++) > (j++)) ? (i++) : (j++)`. With i = 5, j = 3 the program printed **m = 6, i = 7, j = 4**: i was incremented twice and the result is wrong. The function `max_inline(i++, j++)` gave m = 5, i = 6, j = 4 (each argument evaluated exactly once). `SQUARE(i++)` would even be undefined behaviour (two unsequenced modifications).

4.5 When functions are better than macros

- arguments with side effects (i++, function calls such as read_meter()) - evaluated once;
- type checking of arguments and return value - macros accept anything;
- recursion, taking the address of the routine, passing it as a callback;
- multi-statement or long bodies - easier to read and debug (a debugger can step into a function, not into a macro);
- avoiding code bloat when the body is large and used many times.

4.6 Macro vs static inline function (measured: 300,000,000 iterations)

Build	macro SQUARE	static inline	normal (noinline) function
gcc -O0 (time)	0.358 s	0.396 s	0.424 s
gcc -O2 (time)	0.194 s	0.153 s	0.441 s

Execution time: a macro has no call overhead at any optimisation level; `static inline` is a request to the compiler - at -O0 gcc may keep a real call, at -O2 it inlines it and the time equals the macro; an ordinary function pays call/return cost every iteration. **Code size:** macro and inlined code are copied at every use site (grows with number of uses); one ordinary function exists once (smallest code). **Conclusion:** prefer static inline for small calculations - same speed as a macro with type safety and single evaluation; keep macros for constants, token tricks and conditional compilation. *(Timings vary slightly between runs; exact figures are in File 2.)*

TASK 5 - Organise the Program with Header Files and Conditional Compilation (3 marks)

5.1 Structure of a C program (as used here)

Section	Content in this project
1 Documentation	header comments describing each file
2 Preprocessor directives	#include , #include "tariff.h", #define, #ifndef
3 Global declarations	types Customer, Bill; enum Category; const tables; function prototypes (all in tariff.h)
4 main()	main.c - reads lines, validates, bills, prints totals
5 User-defined functions	billing.c - parse_reading, compute_units, energy_charge_paise, generate_bill, ...

Constants and prototypes live in the header `tariff.h`; definitions (the actual code and data) live in `billing.c`. Both main.c and billing.c include the same header so declaration and definition cannot drift apart.

5.2 Include guards

```
#ifndef TARIFF_H
#define TARIFF_H
 ... contents of header ...
#endif /* TARIFF_H */
```

task5_debug.c includes tariff.h **twice** on purpose and compiles without error. Counter-experiment: a header without a guard (noguard.h containing a struct) included twice fails with `error: redefinition of 'struct Point'` (File 2, Task 5 output).

5.3 Conditional debugging

```
#ifdef DEBUG
#define DBG(fmt, ...) fprintf(stderr, "[DEBUG] %s:%d: " fmt "\n", __FILE__, __LINE__, ##__VA_ARGS__)
#else
#define DBG(fmt, ...) ((void)0)
#endif
```

`gcc -DDEBUG` turns the messages on (output shows `[DEBUG] billing.c:82: hh=3001 prev=1000 curr=1180 units=180 energy=31000 ...`); without the flag DBG() expands to nothing and the normal program prints only the bill.

5.4 #include <file> vs #include "file"

Form	Search order	Experiment
#include <tariff.h>	system directories only (/usr/include, gcc's include dir ...) plus -I paths	gcc -c angle.c -> **fatal error: tariff.h: No such file or directory**; with gcc -I. it compiles
#include "tariff.h"	directory of the including file first, then the system paths	compiles without any option

Rule: standard/system headers in angle brackets, the project's own headers in quotes.

5.5 Compile-time debugging vs runtime debugging

Aspect	Compile-time (#ifdef DEBUG, -DDEBUG)	Runtime (if (debug_flag) ...)
Time	zero cost in the release build (code removed)	a test of the flag every time the statement is reached
Executable size	measured text section: 4885 bytes with DEBUG vs 4560 bytes without (debug code only present when asked)	debug code is always present (same size as the DEBUG build)
Flexibility	must recompile to switch on	switch on with ./prog --debug without recompiling (shown in output)
Risk	release build cannot leak debug output	debug path ships to customers

TASK 6 - Read and Print Data Correctly (4 marks)

6.1 Format specifiers

Spec	Type	scanf argument	printf argument	Note
%d	int	∫	int	signed decimal
%u	unsigned int	&unsigned;	unsigned	non-negative decimal
%ld	long	&long;	long	needed on 64-bit Linux for 8-byte long
%lld	long long	&long; long	long long	district totals
%f	float	&float;	**double** (floats are promoted)	printf %f prints a double
%lf	double	&double;	double (the l is ignored)	scanf MUST use %lf for double
%c	char	&char;	char	reads ONE character; use ' %c' to skip whitespace
%s	char string	char[] (no &)	char *	stops at whitespace; give a width (%31s) to avoid overflow

6.2 Reading with scanf, validating with its return value

`scanf` returns the number of fields successfully converted. task6_io.c reads eight fields with one scanf and accepts only if the return value is 8:

```
$ cat inputs/input_valid.txt
1001 12345 600000 8000000000 1.505 2.7182818 D Surya
$ ./task6_io < inputs/input_valid.txt
%d  int           : 1001
%u  unsigned int  : 12345
%ld long          : 600000
%lld long long    : 8000000000
%f  float         : 1.505000   (%.2f -> 1.50)
%lf double (scanf): 2.718282  (printf uses %f/%lf for double)
%c  char          : D
%s  string        : Surya
Amount 12345 paise  ->  Rs 123 and 45 paise  (Rs 123.45)

$ ./task6_io < inputs/input_invalid.txt
INVALID INPUT: scanf converted only 1 of 8 fields -> rejecting
```

6.3 Effect of an incorrect format specifier

```
6:46: warning: format '%d' expects argument of type 'int', but argument 2 has type 'double' [-Wformat=]
7:46: warning: format '%f' expects argument of type 'double', but argument 2 has type 'int' [-Wformat=]
8:46: warning: format '%d' expects argument of type 'int', but argument 2 has type 'long long int' [-Wformat=]
$ ./task6_wrong
printf("%d", 3.14)        -> 759547992
printf("%f", 42)          -> 3.140000
printf("%d", 5000000000LL)-> 705032704   (high bits lost)
printf("%u", -1)          -> 4294967295
printf("%c", 321)         -> A   (321 mod 256 = 65)
```

Compiler warnings (-Wall) catch three of the five; the others compile silently. Result: %d given a double prints garbage (undefined behaviour); %f given an int prints a value taken from an unrelated register (3.14 left from the previous call); %d given a long long keeps only the low 32 bits (5,000,000,000 -> 705,032,704); %u given -1 prints 4294967295; %c given 321 prints 'A'. Wrong specifiers silently corrupt bills - always compile with -Wall -Wformat.

6.4 Printing rupees and paise from an integer amount in paise

```
long paise = 12345;
printf("Rs %ld.%02ld\n", paise / 100, paise % 100);   /* -> Rs 123.45 ; %02ld keeps 5 paise as 05 */
```

6.5 scanf vs fgets() + sscanf() vs fgets() + strtol()

```
input line     | sscanf("%d") (ret,val)     | strtol (value, leftover, errno)
12abc          | ret=1 val=12               | val=12          TRAILING-JUNK errno=0
abc            | ret=0 val=0                | val=0           NO-DIGITS errno=0
   42          | ret=1 val=42               | val=42          clean errno=0
99999999999    | ret=1 val=1215752191       | val=99999999999 clean errno=0
-7             | ret=1 val=-7               | val=-7          clean errno=0

12abc          | ret=1 val=12               | val=12          TRAILING-JUNK errno=0
abc            | ret=0 val=0                | val=0           NO-DIGITS errno=0
   42          | ret=1 val=42               | val=42          clean errno=0
99999999999    | ret=1 val=1215752191       | val=99999999999 clean errno=0
-7             | ret=1 val=-7               | val=-7          clean errno=0
```

Criterion	scanf directly	fgets + sscanf	fgets + strtol
Bad input	failed token stays in the buffer; loops can hang	whole line consumed, so you can re-prompt	whole line consumed
Trailing junk ('12abc')	accepted as 12, junk left behind	accepted as 12 (sscanf stops at 'a')	**detected** via end pointer
Overflow (99999999999)	undefined behaviour / silent wrong value (1215752191 shown)	same	**detected** via errno = ERANGE / range check

Criterion	scanf directly	fgets + sscanf	fgets + strtol
Complexity / cost	O(length), simplest	O(length), extra line buffer (bytes)	O(length), most code, safest

The billing program uses **fgets + sscanf** to split a line, then **strtol**-based validation (parse_reading) for each reading.

TASK 7 - Control Type Conversion and Precision (4 marks)

7.1 Implicit conversion, integer promotion, usual arithmetic conversions

- **Implicit conversion**: the compiler converts automatically when types differ or on assignment (int -> double, long -> int).
- **Integer promotion**: any operand narrower than int (char, short, unsigned short) is first promoted to int. Measured: `unsigned short a = b = 60000; a + b = 120000` (no 16-bit wrap) but `(unsigned short)(a+b)` = 54464; `char 100 + char 100 = 200`, not -56.
- **Usual arithmetic conversions**: for two different arithmetic types both go to the 'higher' rank (int < unsigned < long < long long < float < double). *Signed vs unsigned of the same rank -> unsigned wins*.

7.2 Integer division errors

`7 / 2 = 3`, but `7 / 2.0 = 3.5`; `1 / 3 * 3.0 = 0.0` because 1/3 is integer division = 0. In billing, `(units / 100) * 100` turns 250 units into 200 (slab bug), and average paise = total/units truncates. Fix: multiply first (units * rate) or cast one operand to double / use paise.

7.3 Explicit casting and large long to int

A cast tells the compiler 'convert here'. Narrowing is dangerous: `(int)5000000000L = 705032704` (high 32 bits discarded, measured), `(unsigned char)300 = 44`, `(int)-3.99 = -3` (truncates toward zero). A district total held in long must never be cast to int.

7.4 Rounding fractional charges to paise (33 units x Rs 1.505)

```
rupees (double)         = 49.66499999999999915
(long)(rupees*100)      = 4966   <- truncation (wrong; true value 4966.5 -> 4967)
lround(rupees*100)      = 4967   <- right here only because rupees*100 rounds to 4966.5; not guaranteed
integer: (33*1505+5)/10= 4967   <- exact: keep the rate as 1505 milli-rupees, add half the divisor, then divide
```

Rule: never truncate a floating result into paise; round (half-up) explicitly, preferably in integer arithmetic ((x + d/2) / d).

7.5 Unexpected expression results (measured, task7_conv.c)

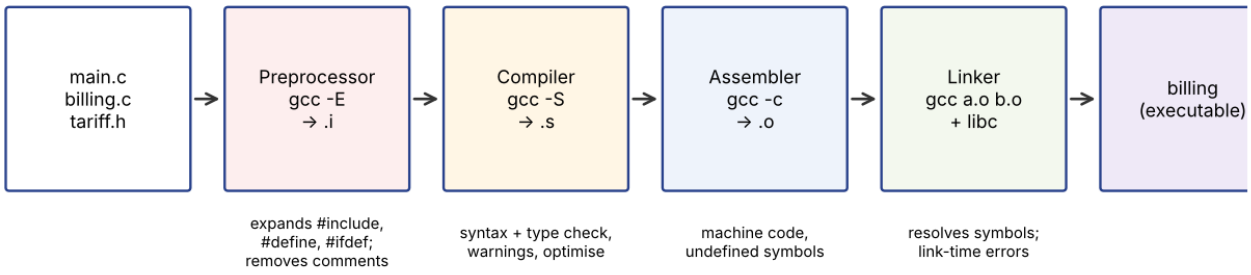
#	Expression	Result	Why
1	(-1 < 1u)	0 (false)	-1 converted to unsigned 4294967295
2	1 / 3 * 3.0	0.0	1/3 is integer division = 0
3	0.1 + 0.2 == 0.3	0 (false)	binary floating-point cannot store 0.1, 0.2, 0.3 exactly
4	ten additions of 0.1f	1.00000012 (== 1.0f false)	accumulated float rounding error
5	sizeof(char) > -1	0 (false)	sizeof is size_t (unsigned): -1 becomes huge
6	5 / 2 * 2.0 vs 5 * 2.0 / 2	4.0 vs 5.0	evaluation order and integer division
7	(int)5000000000L	705032704	narrowing truncates high bits

7.6 Floating-point vs integer-paise calculation

Criterion	float / double	integer paise
Accuracy	measured: 1,000,000 x Rs 0.07 -> float 70266.875 (error +266.88), double 70000.00000019 (error 1.9e-7)	exactly 70000.00 (7,000,000 paise)
Time	FP add 3-5 cycles latency; 0.065 s per 20 M adds	integer add 1 cycle; 0.013 s per 20 M adds (Task 2 run)
Space	float 4 B, double 8 B	int32 4 B (one bill), long 8 B (totals)

TASK 8 - Trace the Compilation Process (4 marks)

Fig. 3 The four stages of C compilation



8.1 The four stages demonstrated (File 2, Task 8 output)

Stage	Command	Observed result
Preprocess	gcc -E main.c -o main.i	main.c has 38 lines; main.i has 1220 lines (tariff.h and stdio.h pasted in); 0 '#define' lines remain; macro names RATE*, DBG, TARIFF_H do not appear
Compile	gcc -S main.c -o main.s	assembly text (.string "Units", ...) - first 12 lines shown in output
Assemble	gcc -c main.c ; gcc -c billing.c	ELF 64-bit relocatable object; nm main.o lists U (undefined) symbols: generate_bill, parse_reading, print_money, printf, fgets ...; nm billing.o lists T symbols (functions it defines)
Link	gcc main.o billing.o -o billing_linked	undefined symbols resolved against billing.o and libc; the executable produces the same bills

8.2 What the preprocessor replaces or removes

- **replaces**: #include lines by the full text of the file; every macro use (RATE2_PAISE -> 200L, SQUARE(x) -> ((x)*(x))); __FILE__ and __LINE__;
- **removes**: all comments, #define lines, and the parts of the source excluded by #ifdef/#ifndef (e.g. DBG becomes ((void)0) when DEBUG is not defined).

8.3 Compile-time, link-time and runtime errors (all three reproduced)

Type	Example file	Message produced
Compile-time	err_compile.c (missing ';')	error: expected ',' or ';' before 'printf'
Link-time	err_link.c (function declared, billing.c not linked)	undefined reference to `energy_charge_paise' / ld returned 1 exit status
Runtime	err_runtime.c (total / units with units = 0)	Floating point exception (SIGFPE), exit status 136; compiles and links without complaint

8.4 Warnings from gcc -Wall -Wextra and how they were resolved

Warning (warn.c, -O2)	Cause	Fix (warn_fixed.c)
comparison of integer expressions of different signedness [-Wsign-compare]	size_t i compared with int n	use int i
unused variable 'unused_var'	declared, never used	delete it
unused parameter 'limit'	parameter not used	use it in the comparison (units[i] > limit)
'c' is used uninitialized	int c; c++	int c = 0;

After the fixes gcc -Wall -Wextra -O2 prints no warnings and the program prints the correct count (2).

8.5 Single-file vs separate compilation

Aspect	Single file / one gcc command	Separate .c -> .o then link
Command	gcc -o billing main.c billing.c	gcc -c main.c ; gcc -c billing.c ; gcc main.o billing.o
Rebuild after editing main.c only	recompiles everything (0.080 s here)	recompiles only main.c and relinks (0.049 s here); gain grows with project size
Disk space	only the executable	extra .o files
Organisation	fine for tiny programs	modules, parallel builds (make -j), reuse of libraries

TASK 9 - Implement and Test the Billing Program (4 marks)

Implementation: main.c, billing.c, tariff.h (File 2, Sections 1-3). All results below were produced by running `./billing < inputs/tests_input.txt` (full output in File 2).

9.1 Pseudocode

```
FUNCTION generate_bill(customer):
    IF household not in 1..999999  RETURN ERR_HOUSEHOLD
    IF category not in 0..2        RETURN ERR_CATEGORY
    IF rollover THEN
        IF curr >= prev RETURN ERR_ROLLOVER_FLAG
        units = 65536 - prev + curr
    ELSE
        IF curr < prev RETURN ERR_DECREASE
        units = curr - prev
    energy = slab_charge(units)       // 100 @ R1, next 100 @ R2, rest @ R3
    fixed  = FIXED[category]
    total  = energy + fixed           // all in paise
    RETURN OK

FUNCTION parse_reading(text):         // validation before any arithmetic
    IF text empty or contains letters/symbols RETURN ERR_FORMAT
    IF text starts with '-'           RETURN ERR_NEGATIVE
    IF value > 65535                  RETURN ERR_RANGE
    RETURN OK
```

9.2 Test table - normal bills across all tariff slabs and five manually calculated bills

HH	Cat	Readings	Units	Manual calculation (paise)	Manual total	Program total	Result
2001	Dom	1000 -> 1050	50	50x150 = 7500; + fixed 5000	12500	12500 (Rs 125.00)	PASS
2002	Dom	1000 -> 1150	150	15000 + 50x200 = 25000; + 5000	30000	30000 (Rs 300.00)	PASS
2003	Dom	1000 -> 1300	300	15000 + 20000 + 100x300 = 65000; + 5000	70000	70000 (Rs 700.00)	PASS
2004	Com	500 -> 620	120	15000 + 20x200 = 19000; + 15000	34000	34000 (Rs 340.00)	PASS
2005	Agri	2000 -> 2250	250	15000 + 20000 + 50x300 = 50000; + 2500	52500	52500 (Rs 525.00)	PASS

9.3 Boundary cases (exactly 100, exactly 200, zero units)

HH	Readings	Units	Expected total	Program total	Result
1006	500 -> 600	100	100x150 + 5000 = 20000	20000	PASS
1009	500 -> 601	101 (just above)	15000 + 200 + 5000 = 20200	20200	PASS
1007	500 -> 700	200	15000 + 20000 + 5000 = 40000	40000	PASS
1010	500 -> 701	201 (just above)	35000 + 300 + 5000 = 40300	40300	PASS
1008	500 -> 500	0	fixed charge only = 5000	5000	PASS

9.4 Meter rollover from 65535 to 0

HH	Readings (rollover = y)	Expected units	Program units / total	Result
1011	65535 -> 0	65536 - 65535 + 0 = 1	1 / 5150 (150 + 5000)	PASS
1012	65500 -> 40	36 + 40 = 76	76 / 16400	PASS
1013	65400 -> 100	136 + 100 = 236	236 / 50800	PASS

9.5 Invalid input (all rejected with a clear message, no crash)

HH	Input	Expected	Program message	Result
1014	prev = abc	reject	reading is not a number (letters/symbols)	PASS
1015	curr = 12x	reject	reading is not a number	PASS
1016	prev = -5	reject	negative reading	PASS
1017	1200 -> 1100, no rollover	reject	current < previous without rollover	PASS
1018	curr = 70000	reject	reading outside 0..65535	PASS
1019	100 -> 200 with rollover = y	reject	rollover flagged but current >= previous	PASS
1020	category 7	reject	invalid category	PASS
-	'bad line here'	reject	malformed line	PASS

Run summary: 13 valid bills, 8 rejected, grand total Rs 3968.50 (396850 paise).

9.6 Fixed slab conditions vs array and loop

Criterion	if / else-if slabs (energy_charge_paise)	array + loop (energy_charge_paise_array)
Time complexity	O(1): at most 3 comparisons	O(k): k = number of slabs (3); still constant for fixed k
Space complexity	O(1)	O(k) for the slab table (3 x 16 bytes)
Measured: 20,000,000 calls	0.037 s	0.071 s
Adding a 4th slab	edit code (new branch, risk of mistakes)	add one table row; loop unchanged
Readability / safety	obvious for 3 slabs	less obvious, but data-driven, ideal for runtime-loaded tariffs
Correctness check	identical results for every unit count 0..1000 (0 mismatches, test_slabs.c)	same

TASK 10 - Compare Expectations with Execution (4 marks)

Program task10_scale.c bills N households with deterministic consumption 0..449 units (all three categories) and sums the totals in five ways.

10.1 float vs integer-paise totals (measured)

Bills	Exact total (int64 paise)	float rupees	float error	double rupees	double error
4,000	2,19,47,800.00 (219478000)	2194780.00	0.00	2194780.0000	0
60,000	3,29,24,559.00 (3292455900)	32923508.00	**-1051.00 Rs**	32924559.0000	0
800,000	43,90,21,194.00 (43902119400)	439074240.00	**+53046.00 Rs**	439021194.0000	0

Float is correct only at 4,000 bills; at 60,000 it is Rs 1,051 wrong and at 800,000 Rs 53,046 wrong - unacceptable for a co-operative's accounts. double was exact at this scale, but only because totals stay below 2^{53}; integer paise is exact by construction.

10.2 Do int, long and long long totals overflow at district scale?

Bills	int (32-bit) total	long (8 B here)	long long
4,000	OK: 219,478,000 paise < 2,147,483,647	219478000	219478000
60,000	**OVERFLOW** (wrapped to -1,002,511,396)	3292455900	3292455900
800,000	**OVERFLOW** (wrapped to 952,446,440)	43902119400	43902119400

INT_MAX = 2,147,483,647 paise = only Rs 2,14,74,836.47, so a 32-bit int total overflows at around 60,000 bills. On this Linux machine long is 64 bits and does not overflow, **but on Windows (or 32-bit systems) long is 32 bits and would overflow exactly like int**. Only long long (or int64_t) is guaranteed 64 bits.

10.3 Actual sizes and ranges vs the minimum guaranteed by the C standard

Type	This system	Standard minimum	Consequence
int	4 B: -2147483648..2147483647	16 bit: -32767..32767	never rely on int holding 100,000
long	8 B: +-9.22e18	32 bit: +-2147483647	portable only to 2.1e9 paise (Rs 2.1 crore)
long long	8 B: +-9.22e18	64 bit: +-9.22e18	safe for district totals everywhere
unsigned short	2 B: 0..65535	16 bit: 0..65535	exactly the meter register

10.4 Expected vs observed - explanation

- Expected: 'a float has plenty of range, so totals are fine'. Observed: range is huge (3.4e38) but **precision** is 24 bits (about 7 digits); above 16,777,216 the spacing between floats exceeds 1, so small bills are rounded away.
- Expected: 'long is big enough'. Observed: true here (8 bytes) but the standard only guarantees 32 bits - the expectation depends on the platform.
- Expected: 'int is 32 bits'. Observed: true here, but only by convention.

10.5 double vs long long for district totals

Criterion	double	long long
Precision	53-bit mantissa: exact integers only up to 2^53 = 9.007e15; measured: 2^53 + 1 stored as ...992 (lost)	exact for every integer up to 9.22e18
Overflow	no overflow (up to 1.8e308) but silently loses accuracy	signed overflow is undefined behaviour above 9.22e18 (not reachable: worst-case district total 4.8e13 paise)
Time (50 M additions)	0.144 s	0.018 s
Space	8 bytes	8 bytes

Worst-case yearly district total used above: 800,000 bills x 12 months x Rs 50,000 = 48,000,000,000,000 paise (4.8e13), far below both 2^{53} and LLONG_MAX, so **long long** is recommended: exact, fast, portable.

TASK 11 - Recommend and Defend the Final Design (4 marks)

11.1 Recommended data types, constants and preprocessor features

Item	Recommendation	Justification
Meter reading	unsigned short (uint16_t)	matches register; wrap-around handled explicitly
Units, per-bill money	long (or int64_t)	exact paise; no overflow in a bill
District/yearly totals	long long / int64_t	guaranteed 64 bit; measured int overflow at ~60,000 bills
Money in general	integer paise, never float	float error Rs 53,046 at 800,000 bills
Tariff constants	#define (overridable with #ifndef + -D), or runtime file	single place of change
Category / status codes	enum	readable, debugger-visible, CAT_COUNT sizes table
Fixed charges	const array	typed, indexed by enum
Preprocessor features	include guards, #ifdef DEBUG, static inline instead of function-like macros	safe re-inclusion, zero-cost debug, no double evaluation

11.2 How tariff revisions are implemented

Rates live only in tariff.h. A revision (e.g. RATE2 150 -> 230) is: change one macro (or pass -DRATE2_PAISE=230), recompile, rerun the test file; the five-bill regression in Task 9 shows the new totals. For revisions without recompilation, tariffs are loaded from tariff.cfg at start-up (implemented in smart_meter.c, Task 12): editing RATE3 from 300 to 350 changed the monthly bill from Rs 641.58 to Rs 676.58 with the same executable.

11.3 Design limitations

- 16-bit register assumes at most one wrap per billing cycle - two wraps would be undetected;
- rollover relies on the meter reader's y/n flag (human error possible);
- compile-time tariffs need a rebuild and redeployment for each revision;
- fixed slab limits (100/200) and one tariff per category; no time-of-day pricing in the original design;
- single-threaded, text input only; no database or audit trail;
- long is only 32 bits on some platforms - needs int64_t for portability.

11.4 Correctness at district scale and experimental evidence

- Every bill is exact (integer paise); the 13 valid + 8 invalid test cases and five hand-calculated bills all pass (Task 9).
- 800,000 bills: float total wrong by Rs 53,046; int wraps negative; long long/int64 exact (Task 10).
- if-else and array-driven slabs agree for all 0..1000 units (0 mismatches).

11.5 Compile-time tariff constants vs tariffs loaded from a configuration file

Criterion	Compile-time constants (#define)	Configuration file
Change without recompiling	no	yes (measured: RATE3 300 -> 350, new bill without rebuild)
Speed	constants folded into the code (fastest)	read once at start-up, then ordinary variables - negligible
Safety	compiler checks values; cannot be corrupted at run time	file may be missing or malformed - must validate (our loader checks all 9 keys)
Auditability	version-controlled with the source	separate file; needs its own versioning
Best for	stable constants (METER_MODULUS, 15 min = 96/day)	tariffs that regulators change

TASK 12 - Re-evaluate and Adapt the Design (8 marks)

New requirement: smart meters send readings in **watt-hours every 15 minutes** (96 per day, **35,040 per household per year**); usage between 6 PM and 10 PM costs **20% more**; tariff values must be **changeable without recompiling**. Implementation: smart_meter.c, tariff.cfg, inputs/readings_day.csv (File 2).

A. Impact analysis (2 marks)

A.1 Changes from the original requirements

Original	New
manual cumulative reading, one per cycle (kWh)	automatic interval readings, 35,040 per household per year (Wh)
unit = kWh, 16-bit register wraps	unit = Wh; energy per interval is a small number, but sums are huge
flat slab rates all day	peak period 18:00-22:00 priced +20%
tariff values compiled in (#define)	tariff values read from a file at run time

A.2 Data types that are now insufficient for watt-hours and yearly totals

- **unsigned short (16-bit)** for cumulative watt-hours: a household using 8,800 Wh/day passes 65,535 Wh in under 8 days - the register would wrap about four times in a 30-day month (264,000 Wh; measured). It is still fine for a *single 15-minute interval* (max 2,500 Wh).
- **int / 32-bit** for district totals: 7.008 x 10^{13} Wh needs 64 bits.
- **int for intermediate charge products** (paise x Wh): 3.15 x 10^{12} exceeds INT_MAX.
- **float** for money or energy: already rejected in Task 10.

A.3 Constants that can no longer remain fixed with #define

RATE1_PAISE, RATE2_PAISE, RATE3_PAISE, SLAB1_LIMIT, SLAB2_LIMIT (now kWh thresholds converted to Wh), fixed charge, and the new peak window (18..22) and surcharge (20%). All are tariff data -> moved to `tariff.cfg`. Constants that stay fixed: 15-minute interval (96/day, 35,040/year) and 1000 Wh = 1 kWh.

B. Re-evaluation of the original design (2 marks)

Original element	Still appropriate?	Reason
integer paise for money	**Yes**	exactness matters even more with 35,040 readings per year
#define for tariff constants	**No**	cannot change without recompiling
unsigned short meter register	**No** for cumulative Wh, yes for per-interval Wh	wraps 4x per month
long for charges	**Partly**	fine on 64-bit Linux but only 32-bit on Windows -> use int64_t
MAX/SQUARE/RUPEES_TO_PAISE macros, include guards, DEBUG	**Yes**	independent of the change
if/else slab code	**Replace** by table/loop	slab limits come from the file; interval-by-interval allocation needed

Largest possible values under the new requirements (measured by smart_meter.c, assumed 10 kW max load, 800,000 households)

Quantity	Calculation	Value	Minimum type
Wh per 15-min reading	10,000 W x 0.25 h	2,500	uint16_t (<= 65,535)
Wh per household per year	2,500 x 35,040	87,600,000	uint32_t (<= 4.29e9), not 16 bit
Wh, whole district per year	87,600,000 x 800,000	70,080,000,000,000 (7.008e13)	**int64_t**
Intermediate paise x Wh x 100 (one household, year, max rate 300, +20%)	87,600,000 x 300 x 120	3,153,600,000,000 (3.15e12)	**int64_t** (INT_MAX = 2.1e9)
Monthly consumption in test (30 days)	264 kWh	264,000 Wh	needs > 16 bit

C. Adaptation / redesign (2 marks)

- **Wider integer types**: int64_t for cumulative Wh, intermediate charge products and all totals; uint16_t kept only for a single interval; money still in integer paise.
- **Runtime tariff loading**: load_tariff() reads tariff.cfg (9 keys: slab limits in kWh, three rates, peak start/end hour, surcharge %, fixed charge); any missing key aborts with an error message.
- **Peak-hour pricing**: each 15-minute reading is priced when it arrives, chronologically, so slab boundaries are honoured inside the month; a reading whose hour is in [18, 22) is priced at (100 + 20)% using integer arithmetic: scaled = normal x 100 + peak x 120; energy charge = (scaled + 50,000) / 100,000 rounded half-up (Wh -> kWh divides by 1000,

percentage by 100).

```
scaled = normal_pwh * 100 + peak_pwh * (100 + t.peak_pct);   /* paise x Wh x 100, int64_t */
energy = (scaled + 50000) / 100000;                          /* to paise, rounded half-up  */
total  = energy + t.fixed_paise;
```

D. Evidence-based defence (2 marks)

Test (30-day month, 96 readings per day)	Result (measured)
readings per day / per year	96 / 35,040 (= 365 x 96)
total consumption	264,000 Wh = 264 kWh (peak 120,000 Wh; off-peak 144,000 Wh)
independent Python check of energy charge	59,158 paise = program value (hand-written reference calculation agrees)
bill with peak rule	energy 59,158 + fixed 5,000 = **64,158 paise = Rs 641.58**
same month without peak rule	energy 54,200 + fixed 5,000 = 59,200 paise -> the peak rule adds 4,958 paise
tariff edited (RATE3 300 -> 350) without recompiling	bill becomes 67,658 paise = Rs 676.58 (same executable)
16-bit register for the same month	would wrap 4 times (264,000 / 65,536)

Trade-offs of the redesign

- + Exactness and no overflow (int64_t) at the price of 8 bytes per value instead of 2: for 35,040 readings per household the raw data is about 280 KB (8 B) vs 70 KB (2 B) - acceptable.
- + Tariffs editable without recompiling, but a bad file can break billing -> every key is validated; a wrong value is now a data error, not a compile error.
- + Runtime loading costs one file read per run (negligible) but loses compiler constant folding; peak rule adds one comparison per reading: 2,880 readings per bill, O(n) time, O(1) extra space.
- + Integer percentage arithmetic keeps exactness; rounding is applied once, at the end, so no accumulated rounding error over 2,880 readings.
- - int64_t is a C99 type; on very old compilers long long would be used instead.

Marks map and viva preparation

Task	Marks	Where in this report	Code / output (File 2)
1 Understand problem and data	4	Task 1 (1.1-1.3)	tariff.h structs
2 Data types	4	Task 2 (2.1-2.5)	task2_types.c + output
3 Constants	3	Task 3 (3.1-3.6)	task3_const.c, billing.c, tariff.h
4 Macros	4	Task 4 (4.1-4.6)	task4_macros.c, macro_expand.c
5 Headers / conditional compilation	3	Task 5 (5.1-5.5)	tariff.h, task5_debug.c, noguard_demo.c
6 Input / output	4	Task 6 (6.1-6.5)	task6_*.c + inputs
7 Conversion and precision	4	Task 7 (7.1-7.6)	task7_conv.c
8 Compilation process	4	Task 8 (8.1-8.5)	run_task8.sh, err_*.c, warn*.c
9 Implement and test	4	Task 9 (9.1-9.6)	main.c, billing.c, tests_input.txt, test_slabs.c
10 Expectations vs execution	4	Task 10 (10.1-10.5)	task10_scale.c
11 Recommend and defend	4	Task 11 (11.1-11.5)	tariff.h, tariff.cfg
12 Smart-meter adaptation	8	Task 12 (A-D)	smart_meter.c, tariff.cfg, readings_day.csv
Total	50		

Questions to be ready for in the viva

- Why paise and not rupees in float? (Task 2 / 10 numbers.)
- Walk through the rollover case 65500 -> 40 = 76 units.
- Why does BAD_SQUARE(2+3) give 11? Why does MAX(i++, j++) misbehave?
- Show me how you would change RATE2 to 230 without editing the source (-DRATE2_PAISE=230).
- What is the difference between a compile-time, link-time and runtime error - show err_*.c.
- Why would int overflow at 60,000 bills, and why may long still fail on Windows?
- How does the peak rule avoid rounding error? (Integer scaled arithmetic, one rounding at the end.)
- Modify the program live: add a fourth slab (use the array version) or change the peak window in tariff.cfg.

Reminder: the GitHub link, output screenshots and the viva are part of the submission requirements. Run run_all.sh and run_task8.sh yourself (File 2 - README) so the screenshots come from your own machine.